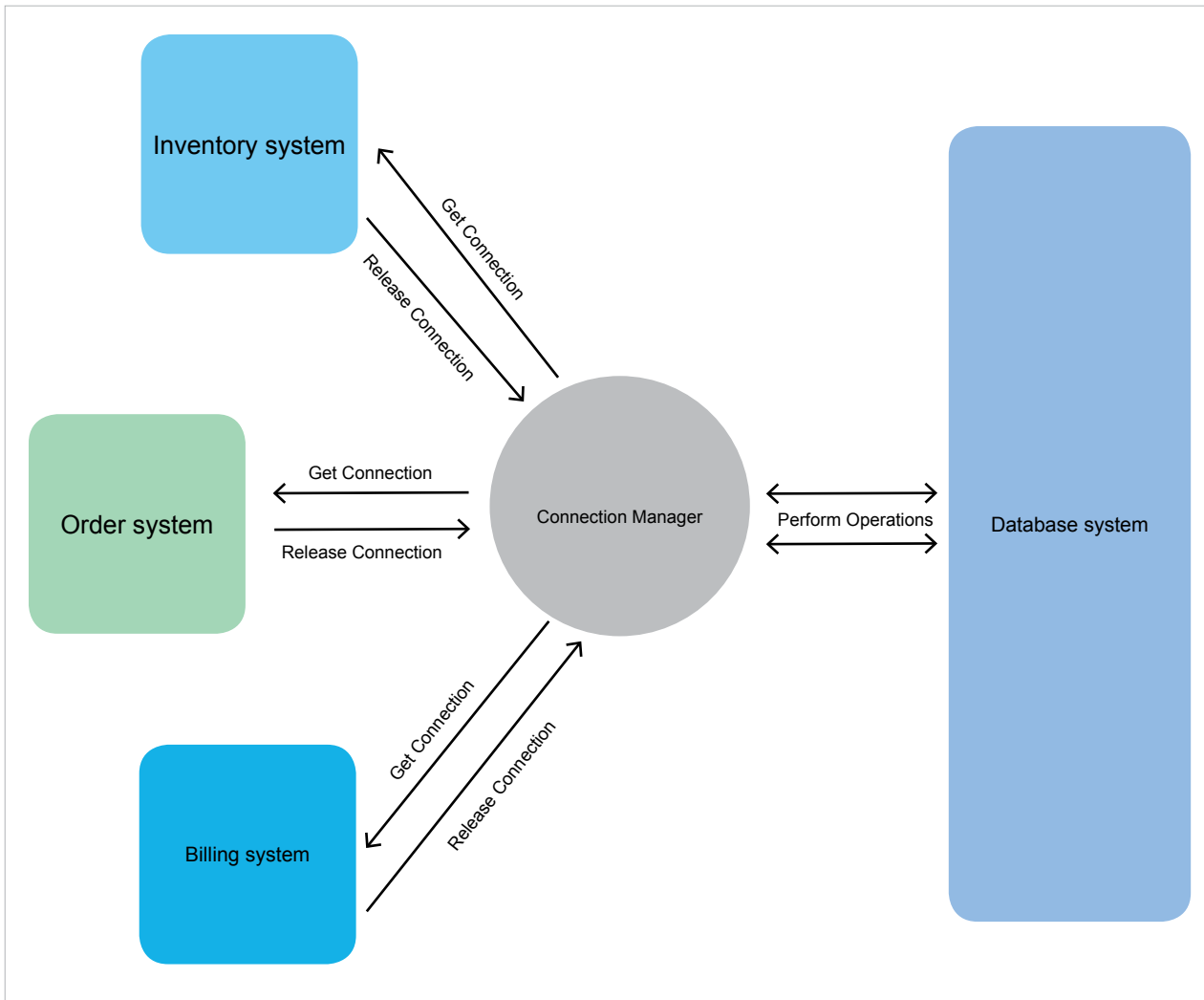

Figure 1: Traditional connections

This approach ensures that the application always has access to a ready-to-use connection without the need to create new connections continuously.

Connection pooling reduces the cost of opening and closing connections by maintaining a 'pool' of open connections that can be passed from one operation to another as needed. This way, we are spared the expense of having to open and close a brand new connection for each operation the system is asked to perform.

Here, I'll demystify connection pools, explain how they work and how to implement them, and explore some of the common issues associated with connection pools. I'll also discuss connection pooling in the cloud and why it's important for modern day applications. By the end of this post, you should have a good understanding of connection pools and how they can help you build more efficient and robust applications.

How connection pools work

The basic principle of connection pooling is to maintain a pool of connections that are ready for use, rather than creating and destroying connections as required. When a client requests a connection from the pool, the connection pool manager checks if there are any available connections in the pool. If one exists, the connection pool manager returns the connection to the client. Otherwise, the connection pool manager creates a new connection, adds it to the pool, and returns the new connection to the client.

Connection pooling algorithms are used to manage the pool of connections. These algorithms determine when to create new connections and when to reuse existing connections. The most common algorithms used for connection pooling are LRU (least recently used), and round-robin or FIFO (first in, first out).